

Relationships Between Entities in Entity Framework Core

In this article, I will discuss the Relationships Between Entities in the Entity Framework Core. At the end of this article, you will understand how the entity framework manages the relationships between entities. Please read our previous article discussing [Fluent API Configurations in Entity Framework Core](#) with Examples.

What are Relationships?

In relational databases, relationships between tables (or entities) define how data in one table relates to data in another table. Establishing these relationships is essential for maintaining data integrity and enabling meaningful data retrieval. When we create relationships between two entities in Entity Framework Core, one entity serves as the **Principal Entity**, and the other entity serves as the **Dependent Entity**.

- **Principal Entity:** This entity contains the primary or unique key, which other entities depend on.
- **Dependent Entity:** This entity contains the foreign key that references the primary or unique key of the principal entity.

Understanding the principal and dependent entities helps EF Core manage data operations correctly, especially when inserting or deleting related data.

Key Terms in a Relationship:

- **Primary Key (PK):** A column or a set of columns (in the case of a composite primary key) uniquely identifying each row in a table.
- **Foreign Key (FK):** A column or a set of columns (in the case of a composite foreign key) in one table that references the primary key or unique key of another table, thus establishing a relationship between them.
- **Navigation Properties:** Navigation Properties in entity classes allow users to navigate from one entity to a related entity or collection of entities.

Types of Navigation Properties:

In Entity Framework core, Navigation Properties represent the relationships between entities. Depending on the type of relationships between the entities, navigation properties can exist on either the principal or dependent entity or on both entities. There are two types of Navigation Properties. They are as follows:

- **Reference Navigation Property:** It represents a single related entity, typically in a one-to-one relationship. For example, a Customer might have exactly one Address.
- **Collection Navigation Property:** It represents a collection of related entities commonly used in one-to-many or many-to-many relationships. For example, a Customer might have many Orders.

Types of Relationships in Database:

In relational databases, relationships refer to how tables are connected based on shared data elements. The main types are:

- **One-to-One (1:1) Relationship:** Each record in one table is associated with one and only one record in another table. This relationship is used when two entities have a unique and direct link. For example, a student can have only one address, and each address can belong to a single student.
- **One-to-Many (1: M) Relationship:** A single record in one table is related to multiple records in another table. This is the most common type of relationship, where one entity (e.g., Customer) can have many related entities (e.g., Orders), but each Order belongs to only one Customer.
- **Many-to-Many (M: M) Relationship:** Multiple records in one table are related to multiple records in another table. To implement this, we use a join table that contains foreign keys from both related tables. For example,

Students can be enrolled in many Courses, and each Course can have many Students.

- **Self-Referencing Relationship:** A table can have a relationship with itself. For example, in an organization, an employee might be related to a manager who is also an employee.

Types of Relationships in Entity Framework Core:

In Entity Framework Core, the types of relationships are similar to those in relational databases, but they are represented using object-oriented principles, navigation properties, and foreign key constraints. EF Core supports the following relationships:

1. **One-to-One Relationship**
2. **One-to-Many Relationship**
3. **Many-to-Many Relationship**
4. **Self-Referencing Relationship**

One-to-One (1:1) Relationship in EF Core:

In a one-to-one relationship, each entity points to only one instance of the other entity. EF Core establishes this relationship when both entities have reference navigation properties pointing to each other. A foreign key constraint may be used on one of the entities. Both related columns are primary keys in the database or have unique constraints, ensuring a one-to-one link. The following is the Implementation Guidelines in EF Core for One-to-One Relationship:

- Both entities have reference navigation properties pointing to each other.
- The dependent entity's primary key often acts as a foreign key to the principal entity.
- Use data annotations like [ForeignKey] or Fluent API configurations to define the relationship explicitly.

One-to-One Relationship Example using EF Core Data Annotation Attribute:

```
public class User
{
    public int UserId { get; set; }
    public string Username { get; set; }
    public UserProfile Profile { get; set; } // Reference navigation property
}
public class UserProfile
{
    [Key]
    ForeignKey("User")
    public int UserId { get; set; } // Primary key and foreign key
    public string Bio { get; set; }
    public User User { get; set; } // Reference navigation property
}
```

Here:

- The UserProfile entity uses UserId as its primary key and also a foreign key pointing to the User entity.
- Navigation properties in both entities facilitate navigation between them.

One-to-Many (1:M) Relationship in EF Core:

In a one-to-many relationship, one entity instance (the principal) relates to multiple instances of another entity (the dependents). Typically, the principal entity holds the primary key, while the dependent entity holds the foreign key. The

following is the Implementation Guidelines in EF Core for One-to-Many Relationship:

- The principal entity includes a collection navigation property.
- The dependent entity includes a reference navigation property and a foreign key.

One-to-Many Relationship Example in EF Core:

```
public class Customer
{
    public int CustomerId { get; set; }
    public string Name { get; set; }
    public ICollection<Order> Orders { get; set; } // Collection navigation
property
}
public class Order
{
    public int OrderId { get; set; }
    public DateTime OrderDate { get; set; }
    public int CustomerId { get; set; } // Foreign key
    public Customer Customer { get; set; } // Reference navigation property
}
```

Here:

- The Order entity has a CustomerId foreign key and a Customer reference navigation property.
- The Customer entity has an Orders collection navigation property.
- EF Core can handle this relationship automatically by convention based on the navigation properties and the foreign key.

Many-to-Many (M:M) Relationship in EF Core:

In EF Core 5.0 and later, many-to-many relationships are supported without requiring an explicit join entity. In this relationship, both entities have collection navigation properties, and EF Core creates an implicit join table in the database to link the two. The following is the Implementation Guidelines in EF Core for One-to-Many Relationship:

Many-to-Many Relationship Example in EF Core:

```
public class Student
{
    public int StudentId { get; set; }
    public string Name { get; set; }
    public ICollection<Course> Courses { get; set; } // Collection navigation
property
}
public class Course
{
    public int CourseId { get; set; }
    public string Title { get; set; }
    public ICollection<Student> Students { get; set; } // Collection
navigation property
}
```

Behind the scenes, EF Core will create a join table for StudentCourse containing the foreign keys StudentId and CourseId.

Self-Referencing Relationship in EF Core:

In EF Core, a self-referencing relationship occurs when an entity relates to other instances of the same entity. This is useful for hierarchical data like organizational structures (e.g., Employee and Manager relationships). In EF Core, we can configure this using the [ForeignKey] attribute or the Fluent API.

Self-Referencing Relationship Example in EF Core:

```
public class Employee
{
    public int EmployeeId { get; set; }
    public string Name { get; set; }
    public int? ManagerId { get; set; } // Foreign key to another Employee
    public Employee Manager { get; set; } // Reference navigation property
    public ICollection<Employee> Subordinates { get; set; } // Collection
navigation property
}
```

Here:

- The Employee entity includes a ManagerId foreign key referencing another Employee.
- The Manager property allows navigation to the manager, and Subordinates allow navigation to employees managed by this employee.
- EF Core can handle self-referencing relationships by convention but may require Fluent API for complex configurations.

Required vs. Optional Relationships:

- **Required Relationship:** The foreign key cannot be null; the dependent must have a principal.
- **Optional Relationship:** The foreign key can be null; the dependent may or may not have a principal.

In the next article, I will discuss [How to Configure One-to-One Relationships in Entity Framework Core using Fluent API](#) with Examples. In this article, I explain the basic concepts of **Relationships between Entities in the Entity Framework Core**. I hope you enjoyed this Relationship in EF Core Fluent API article.